МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА ВТ

ОТЧЁТ

по лабораторной работе №2

Тема: Методы информированного (эвристического) поиска

Студент гр. 3312	_____	Барченков П.А.
Студент гр. 3312	_____	Лебедев И.А.
Студент гр. 3312	_____	Шарапов И.Д.
Преподаватель	_____	Пантелеев М.Г.

Санкт-Петербург

2026

Оглавление

Цель работы 3

Постановка задачи (Вариант 1) 3

Описание выбранных структур данных, представление функции определения последователей 4

Описание алгоритма 5

Исходный код 6

Результат работы 12

Вывод 17

Цель работы

Практическое закрепление теоретических основ информированного (эвристического) поиска.

Постановка задачи (Вариант 1)

Во второй лабораторной работе рассматривается задача решения головоломки 8-ки с применением методов информированного поиска. Игровое поле представляет собой квадрат 3×3, на котором расположены восемь пронумерованных фишек и одна пустая клетка, обозначаемая числом 0. За один ход разрешается переместить в пустую клетку одну из соседних по вертикали или горизонтали фишек.

Для варианта 1 требуется найти последовательность ходов, переводящую начальное состояние 583402761 в целевое состояние 123456780. Состояния в программе представлены в виде девятизначного числа, где каждая цифра соответствует содержимому одной позиции поля. Поиск решения выполняется в пространстве состояний, где вершинами являются различные конфигурации игрового поля, а переходами – допустимые перемещения пустой клетки.

Для решения задачи необходимо реализовать методы информированного поиска: алгоритм A* и жадный поиск. В качестве эвристических функций используются:

- h1 – число фишек, стоящих не на своих местах
- h2 – сумма манхэттенских расстояний фишек до их целевых позиций

В ходе выполнения работы требуется реализовать указанные алгоритмы, обеспечить выбор эвристической функции и режима работы программы, получить решение задачи и определить экспериментальные оценки временной и ёмкостной сложности, а также сравнить эффективность алгоритма A* и жадного поиска при использовании эвристик h1 и h2.

Описание выбранных структур данных, представление функции определения последователей

В программе состояние головоломки представляется в виде девятизначного целого числа, где каждая цифра соответствует содержимому одной клетки игрового поля 3×3. Начальное состояние задаётся константой START_STATE = 583402761, а целевое – GOAL_STATE = 123456780. Такой способ представления позволяет компактно хранить конфигурацию поля и выполнять преобразования без использования массивов.

Для обращения к отдельным элементам состояния используется функция extract_digit(x, i), возвращающая цифру в позиции i. Формирование нового состояния после перемещения фишки выполняется функцией swap_digits(x, i, j), которая меняет местами пустую клетку и соседнюю с ней фишку.

Допустимые переходы между позициями задаются словарём pos_from_pos. Для каждой клетки в нём указан список соседних клеток, в которые может быть перемещена пустая клетка. Например, для центральной позиции 4 допустимы переходы в позиции 1, 3, 5 и 7.

Функция определения последователей реализована как gen_children(x). Она находит положение пустой клетки и по словарю pos_from_pos формирует список всех состояний, которые могут быть получены из текущего за один допустимый ход.

Для реализации информированного поиска используется приоритетная очередь heapq. В ней хранятся открытые вершины, ожидающие раскрытия. В жадном поиске приоритетом является значение эвристической функции h, а в алгоритме A* – сумма g + h, где g – стоимость пути от начального состояния до текущей вершины, а h – эвристическая оценка расстояния до цели.

Для хранения информации о найденных состояниях используется словарь history, в котором для каждой вершины сохраняется её родитель. Это позволяет после нахождения цели восстановить путь от конечного состояния к начальному. Стоимость пути до каждой вершины хранится в словаре g_costs, где каждому состоянию соответствует число ходов от начального состояния.

Для исключения повторного раскрытия уже обработанных состояний используется множество closed. Если вершина уже содержится в этом множестве, повторно она не раскрывается.

В программе реализованы две эвристические функции. Функция h1(state) вычисляет количество фишек, находящихся не на своих местах, не учитывая пустую клетку. Функция h2(state) вычисляет сумму манхэттенских расстояний фишек до их целевых позиций. Для вычисления эвристики h2 используется словарь GOAL_POS, содержащий позиции цифр в целевом состоянии.

Дополнительно в программе реализована функция is_solvable(start, goal), позволяющая до запуска поиска проверить разрешимость задачи.

Описание алгоритма

В программе реализованы два метода информированного поиска: алгоритм A* и жадный поиск. Оба алгоритма работают в пространстве состояний головоломки 8-ки, где вершинами являются конфигурации игрового поля, а переходами – допустимые перемещения пустой клетки.

В начале работы выполняется проверка разрешимости задачи с помощью функции is_solvable(start, goal). Если начальное состояние неразрешимо для заданного целевого состояния, поиск не запускается.

После этого начальное состояние помещается в приоритетную очередь. Для него в словаре history сохраняется ссылка на самого себя, а в словаре g_costs записывается начальная стоимость пути, равная нулю. Далее выполняется основной цикл поиска: из приоритетной очереди извлекается вершина с наименьшим значением приоритета. Если извлечённое из очереди состояние уже содержится в множестве closed, оно не раскрывается повторно. В противном случае вершина заносится в множество closed, после чего для неё строятся все допустимые потомки с помощью функции gen_children(x).

Если текущая вершина совпадает с целевым состоянием, поиск завершается. После этого программа восстанавливает путь от цели к

начальному состоянию, последовательно переходя по словарю history от каждой вершины к её родителю.

Для каждого дочернего состояния вычисляется новая стоимость пути g, равная стоимости пути до текущей вершины плюс один ход. Если потомок ещё не встречался ранее или для него найден более короткий путь, в словарях history и g_costs обновляется информация о родителе и стоимости пути, после чего вершина добавляется в приоритетную очередь.

Различие между алгоритмами A* и жадным поиском определяется способом вычисления приоритета. В жадном поиске приоритетом является только значение эвристической функции h, поэтому выбирается вершина, которая по эвристической оценке кажется ближайшей к цели. В алгоритме A* приоритет вычисляется как сумма g + h, где g – стоимость пути от начального состояния до текущей вершины, а h – эвристическая оценка расстояния до цели. Благодаря этому A* учитывает как уже пройденный путь, так и предполагаемое расстояние до цели.

В программе используются две эвристики. Эвристика h1 определяет количество фишек, стоящих не на своих местах, не учитывая пустую клетку. Эвристика h2 вычисляет сумму манхэттенских расстояний фишек до их целевых позиций. Это позволяет сравнить эффективность алгоритмов при использовании разных способов оценки близости состояния к цели.

Исходный код

```
import heapq
import os

pos_from_pos = {0: [1, 3],
                1: [0, 2, 4],
                2: [1, 5],
                3: [0, 4, 6],
                4: [1, 3, 5, 7],
                5: [2, 4, 8],
                6: [3, 7],
                7: [4, 6, 8],
                8: [5, 7]
                }

START_STATE = 583402761
GOAL_STATE = 123456780
```

```python
BLOCK_WIDTH = 8
TREE_LINE_SHIFT = 2


def clear_console():
    os.system('cls' if os.name == 'nt' else 'clear')


def extract_digit(x: int, i: int) -> int:
    return x % 10 ** (9 - i) // 10 ** (8 - i)


def build_goal_pos(goal: int) -> dict:
    goal_pos = {}
    for i in range(9):
        goal_pos[extract_digit(goal, i)] = i
    return goal_pos

GOAL_POS = build_goal_pos(GOAL_STATE)

def swap_digits(x: int, i: int, j: int) -> int:
    di = extract_digit(x, i)
    dj = extract_digit(x, j)
    return x + (dj - di) * 10 ** (8 - i) + (di - dj) * 10 ** (8 - j)


def gen_children(x: int):
    zero_pos = 0
    for i in range(9):
        if extract_digit(x, i) == 0:
            zero_pos = i
            break
    ans = []
    for j in pos_from_pos[zero_pos]:
        ans.append(swap_digits(x, zero_pos, j))
    return ans


def state_row_text(state: int, row: int) -> str:
    return ''.join(f'{extract_digit(state, row * 3 + j)} ' for j in range(3)) + '  '


def grid_print(ans: list, step=-1):
    for i in range(3):
        for k in range(len(ans)):
            print(state_row_text(ans[k], i), end='')
        if step != -1 and i == 1:
            print(f'Шаг {step}')
        else:
            print()


def h1(state: int) -> int:
    cnt = 0
    for i in range(9):
        digit = extract_digit(state, i)
        if digit != 0 and GOAL_POS[digit] != i:
            cnt += 1
    return cnt
```

```python
def h2(state: int) -> int:
    dist = 0
    for i in range(9):
        digit = extract_digit(state, i)
        if digit != 0:
            goal_pos = GOAL_POS[digit]
            dist += abs(i // 3 - goal_pos // 3) + abs(i % 3 - goal_pos % 3)
    return dist


def calc_h(state: int, heuristic: str) -> int:
    return h1(state) if heuristic == 'h1' else h2(state)


def score_lines_for_state(state: int, g: int, heuristic: str, alg: str):
    h = calc_h(state, heuristic)
    if alg == 'A*':
        return [f' g={g}', f' h={h}', f' f={g + h}']
    return [f' h={h}']


def print_root_with_scores(state: int, g: int, heuristic: str, alg: str):
    scores = score_lines_for_state(state, g, heuristic, alg)
    for i in range(3):
        right = scores[i] if i < len(scores) else ''
        print(state_row_text(state, i) + '    ' + right)


def print_children_tree(children: list, g_costs: dict, parent_state: int, closed: set,
heuristic: str, alg: str):
    count = len(children)
    if count == 0:
        return

    total_width = count * BLOCK_WIDTH
    centers = [max(0, i * BLOCK_WIDTH + BLOCK_WIDTH // 2 - TREE_LINE_SHIFT) for i in
range(count)]
    first_center = centers[0]
    last_center = centers[-1]

    print('  |')

    horiz = [' '] * total_width
    for i in range(first_center, last_center + 1):
        horiz[i] = '-'
    print(''.join(horiz).rstrip())

    pipes = [' '] * total_width
    for c in centers:
        pipes[c] = '|'
    print(''.join(pipes).rstrip())

    for row in range(3):
        parts = [state_row_text(child, row).ljust(BLOCK_WIDTH) for child in children]
        print(''.join(parts).rstrip())

    status = [' '] * total_width
    for idx, child in enumerate(children):
        child_g = g_costs[parent_state] + 1
        repeated = child in closed or (child in g_costs and child_g >= g_costs[child])
        if repeated:
```

```
            status[centers[idx]] = '✓'
    print(''.join(status).rstrip())

    rows = 3 if alg == 'A*' else 1
    for row in range(rows):
        parts = []
        for child in children:
            child_g = g_costs[parent_state] + 1
            scores = score_lines_for_state(child, child_g, heuristic, alg)
            txt = scores[row] if row < len(scores) else ''
            parts.append(txt.ljust(BLOCK_WIDTH))
        print(''.join(parts).rstrip())


def print_state_with_scores(state: int, g: int, heuristic: str, alg: str, step: int):
    grid_print([state], step)
    h = calc_h(state, heuristic)
    if alg == 'A*':
        print(f'g={g}, h={h}, f={g + h}')
    else:
        print(f'h={h}')


def trek_print(ans: list, g_costs: dict, heuristic: str, alg: str):
    path = ans[::-1]
    for i in range(len(path)):
        print_state_with_scores(path[i], g_costs[path[i]], heuristic, alg, i)
        if i != len(path) - 1:
            print('  |\n  V')


def restore_path(parents: dict, start: int, goal: int):
    ans = []
    u = goal
    while u != start:
        ans.append(u)
        u = parents[u]
    ans.append(u)
    return ans


def is_solvable(start: int, goal: int) -> bool:
    start_digits = [extract_digit(start, i) for i in range(9) if extract_digit(start,
i) != 0]
    goal_digits = [extract_digit(goal, i) for i in range(9) if extract_digit(goal, i)
!= 0]
    start_inv = 0
    goal_inv = 0
    for i in range(len(start_digits)):
        for j in range(i + 1, len(start_digits)):
            if start_digits[i] > start_digits[j]:
                start_inv += 1
    for i in range(len(goal_digits)):
        for j in range(i + 1, len(goal_digits)):
            if goal_digits[i] > goal_digits[j]:
                goal_inv += 1
    return start_inv % 2 == goal_inv % 2


def solve(start: int, goal: int, alg='A*', heuristic='h1', mode='auto'):
    if not is_solvable(start, goal):
```

```
        return None, 0, 0, 0

    open_heap = []
    history = {start: start}
    g_costs = {start: 0}
    closed = set()
    push_cnt = 0
    time_compl = 0

    start_h = calc_h(start, heuristic)
    heapq.heappush(open_heap, (start_h, push_cnt, start))

    while len(open_heap) > 0:
        _, _, state = heapq.heappop(open_heap)
        if state in closed:
            continue

        closed.add(state)
        time_compl += 1
        children = gen_children(state)

        if mode == 'manual':
            print_root_with_scores(state, g_costs[state], heuristic, alg)
            print_children_tree(children, g_costs, state, closed, heuristic, alg)
            print()
            if input() == '0':
                return None, 0, 0, 0

        if state == goal:
            ans = restore_path(history, start, state)
            return ans, g_costs, time_compl, len(history.keys())

        for child in children:
            if child in closed:
                continue
            new_g = g_costs[state] + 1
            if child not in g_costs or new_g < g_costs[child]:
                history[child] = state
                g_costs[child] = new_g
                push_cnt += 1
                child_h = calc_h(child, heuristic)
                priority = child_h if alg == 'Greedy' else new_g + child_h
                heapq.heappush(open_heap, (priority, push_cnt, child))

    return None, 0, 0


while True:
    clear_console()
    print('Начальное состояние:')
    grid_print([START_STATE])
    print('Целевое состояние:')
    grid_print([GOAL_STATE])
    print('Выберите алгоритм решения:\n'
          '1. A*\n'
          '2. Жадный поиск\n\n'
          '0. Выйти')
    x = input()
    while x not in ['0', '1', '2']:
        clear_console()
        print('Некорректный ввод!')
```

```
    print('Выберите алгоритм решения:\n'
          '1. A*\n'
          '2. Жадный поиск\n\n'
          '0. Выйти')
    x = input()

if x == '1':
    alg = 'A*'
elif x == '2':
    alg = 'Greedy'
else:
    break

clear_console()
print(f'Алгоритм "{alg}".')
print('Выберите эвристическую функцию:\n'
      '1. h1 — число фишек не на своем месте\n'
      '2. h2 — манхэттенское расстояние\n\n'
      '0. Выйти')
x = input()
while x not in ['0', '1', '2']:
    clear_console()
    print('Некорректный ввод!')
    print(f'Алгоритм "{alg}".')
    print('Выберите эвристическую функцию:\n'
          '1. h1 — число фишек не на своем месте\n'
          '2. h2 — манхэттенское расстояние\n\n'
          '0. Выйти')
    x = input()

if x == '1':
    heuristic = 'h1'
elif x == '2':
    heuristic = 'h2'
else:
    break

clear_console()
print(f'Алгоритм "{alg}", эвристика "{heuristic}".')
print('Выберите режим работы:\n'
      '1. Пошаговый\n'
      '2. Автоматический\n\n'
      '0. Выйти')
x = input()
while x not in ['0', '1', '2']:
    clear_console()
    print('Некорректный ввод!')
    print(f'Алгоритм "{alg}", эвристика "{heuristic}".')
    print('Выберите режим работы:\n'
          '1. Пошаговый\n'
          '2. Автоматический\n\n'
          '0. Выйти')
    x = input()

if x == '1':
    mode = 'manual'
elif x == '2':
    mode = 'auto'
else:
    break
```

```
    clear_console()
    if mode == 'manual':
        print(f'Алгоритм "{alg}", эвристика "{heuristic}", режим "Пошаговый".')
        print('Для следующего шага нажмите любую клавишу, кроме "0"')
        print('Чтобы вернуться в меню, нажмите "0".')
        input()
        solve(START_STATE, GOAL_STATE, alg, heuristic, mode)
    else:
        print(f'Алгоритм "{alg}", эвристика "{heuristic}", режим "Автоматический".\n')
        ans, g_costs, time_compl, space_compl = solve(START_STATE, GOAL_STATE, alg,
heuristic, mode)
        if ans is None:
            print('Решение не найдено.')
        else:
            trek_print(ans, g_costs, heuristic, alg)
            print('---------------------------------------------')
            print(f'Ёмкостная сложность: {space_compl}')
            print(f'Временная сложность: {time_compl}')
            print(f'Стоимость пути: {len(ans) - 1}')
            print('---------------------------------------------')
        print('Чтобы вернуться в меню, нажмите любую клавишу.')
        input()
```

Результат работы

В результате выполнения программы были получены решения головоломки 8-ки с использованием двух методов информированного поиска: алгоритма A* и жадного поиска. Программа позволяет последовательно выбрать алгоритм решения, эвристическую функцию и режим работы. В пошаговом режиме отображается процесс раскрытия вершин и вычисления оценок состояний, а в автоматическом режиме выводится найденный путь от начального состояния к целевому, а также экспериментальные оценки ёмкостной и временной сложности и стоимость пути.

На рисунках ниже представлены основные этапы работы программы: выбор алгоритма, выбор эвристической функции, выбор режима работы, пример пошагового выполнения, а также результаты автоматического режима для всех сочетаний алгоритмов и эвристик.

```
Выберите алгоритм решения:
1. A*
2. Жадный поиск

0. Выйти
```

Рисунок 1 – Выбор алгоритма решения задачи

```
Выберите эвристическую функцию:
1. h1 – число фишек не на своем месте
2. h2 – манхэттенское расстояние

0. Выйти
```

Рисунок 2 – Выбор эвристической функции

```
Выберите режим работы:
1. Пошаговый
2. Автоматический

0. Выйти
```

Рисунок 3 – Выбор режима работы программы

```
Алгоритм "A*", эвристика "h1", режим "Пошаговый".
Для следующего шага нажмите любую клавишу, кроме "0"
Чтобы вернуться в меню, нажмите "0".

5 8 3       g=0
4 0 2       h=5
7 6 1       f=5
  |
  -------------------------
  |       |       |       |
5 0 3   5 8 3   5 8 3   5 8 3
4 8 2   0 4 2   4 2 0   4 6 2
7 6 1   7 6 1   7 6 1   7 0 1

 g=1     g=1     g=1     g=1
 h=5     h=6     h=5     h=5
 f=6     f=7     f=6     f=6


5 0 3       g=1
4 8 2       h=5
7 6 1       f=6
  |
  -----------------
  |       |       |
0 5 3   5 3 0   5 8 3
4 8 2   4 8 2   4 0 2
7 6 1   7 6 1   7 6 1
                  ✓
 g=2     g=2     g=2
 h=5     h=6     h=5
 f=7     f=8     f=7


5 8 3       g=1
4 2 0       h=5
7 6 1       f=6
  |
  -----------------
  |       |       |
5 8 0   5 8 3   5 8 3
4 2 3   4 0 2   4 2 1
7 6 1   7 6 1   7 6 0
          ✓
 g=2     g=2     g=2
 h=6     h=5     h=5
 f=8     f=7     f=7
```

Рисунок 4 – Пример работы программы в пошаговом режиме для A* с h1

```
  |
  V
1 2 3
4 0 6   Шаг 42
7 5 8
h=2
  |
  V
1 2 3
4 5 6   Шаг 43
7 0 8
h=1
  |
  V
1 2 3
4 5 6   Шаг 44
7 8 0
h=0
---------------------------------------------
Ёмкостная сложность: 1375
Временная сложность: 850
Стоимость пути: 44
---------------------------------------------
Чтобы вернуться в меню, нажмите любую клавишу.
```

Рисунок 5 – Результат работы программы в автоматическом режиме для жадного поиска с эвристикой h1

```
  |
  V
1 2 3
4 5 6   Шаг 44
0 7 8
h=2
  |
  V
1 2 3
4 5 6   Шаг 45
7 0 8
h=1
  |
  V
1 2 3
4 5 6   Шаг 46
7 8 0
h=0
---------------------------------------------
Ёмкостная сложность: 648
Временная сложность: 388
Стоимость пути: 46
---------------------------------------------
Чтобы вернуться в меню, нажмите любую клавишу.
```

Рисунок 6 – Результат работы программы в автоматическом режиме для жадного поиска с эвристикой h2

```
  |
  V
1 2 3
4 5 6   Шаг 20
0 7 8
g=20, h=2, f=22
  |
  V
1 2 3
4 5 6   Шаг 21
7 0 8
g=21, h=1, f=22
  |
  V
1 2 3
4 5 6   Шаг 22
7 8 0
g=22, h=0, f=22
----------------------------------------------
Ёмкостная сложность: 15557
Временная сложность: 10152
Стоимость пути: 22
----------------------------------------------
Чтобы вернуться в меню, нажмите любую клавишу.
```

Рисунок 7 – Результат работы программы в автоматическом режиме для алгоритма A* с эвристикой h1

```
  |
  V
1 2 3
4 5 6   Шаг 20
0 7 8
g=20, h=2, f=22
  |
  V
1 2 3
4 5 6   Шаг 21
7 0 8
g=21, h=1, f=22
  |
  V
1 2 3
4 5 6   Шаг 22
7 8 0
g=22, h=0, f=22
----------------------------------------------
Ёмкостная сложность: 2267
Временная сложность: 1442
Стоимость пути: 22
----------------------------------------------
Чтобы вернуться в меню, нажмите любую клавишу.
```

Рисунок 8 – Результат работы программы в автоматическом режиме для алгоритма A* с эвристикой h2

Вывод

В ходе выполнения работы были изучены и реализованы два метода информированного поиска: жадный поиск и алгоритм A* для решения головоломки 8-ки. В отличие от первой лабораторной работы, здесь при выборе следующей вершины использовалась не только структура пространства состояний, но и эвристическая оценка близости состояния к цели. В программе были реализованы две эвристики: h1 – число фишек, стоящих не на своих местах, и h2 – сумма манхэттенских расстояний до целевых позиций.

Практические результаты показали, что алгоритм A* даёт более качественное решение, чем жадный поиск. Для обеих эвристик алгоритм A* нашёл путь стоимостью 22 шага, тогда как жадный поиск дал путь стоимостью 44 шага для h1 и 46 шагов для h2. Это объясняется тем, что жадный поиск ориентируется только на эвристическую оценку текущего состояния, а алгоритм A* учитывает как эвристику, так и стоимость уже пройденного пути. Поэтому A* обеспечивает более выгодный итоговый маршрут.

Сравнение эвристик показало, что эвристика h2 в алгоритме A* работает эффективнее, чем h1. При одинаковой стоимости пути 22 шага A* с h2 потребовал существенно меньше памяти и времени, чем A* с h1. Для жадного поиска эвристика h2 также уменьшила временную и ёмкостную сложность, однако не улучшила стоимость найденного пути. Таким образом, жадный поиск оказался наиболее экономичным по времени и памяти, тогда как A* с эвристикой h2 показал наилучший результат по качеству решения при значительно меньших затратах по сравнению с BFS и A* с эвристикой h1.

Для итогового сравнения результатов лабораторных работ 1 и 2 приведена таблица 1.

Таблица 1

Оценка эффективности стратегий	Неинформированный поиск		Информированный поиск			
	BFS	DFS	Жадный поиск		Поиск A*	
			h1	h2	h1	h2
Стоимость пути	22	48436	44	46	22	22
Временная сложность	90532	55404	850	388	10152	1442
Ёмкостная сложность	114058	97731	1375	648	15557	2267

По данным таблицы 1 видно, что среди методов неинформированного поиска BFS обеспечивает наилучшее качество решения, так как находит кратчайший путь, однако требует наибольших затрат памяти и времени. DFS оказался менее затратным по экспериментальным оценкам, но привёл к крайне невыгодному пути, поэтому для данной задачи его применение неэффективно.

Среди методов информированного поиска жадный поиск показывает наименьшие затраты по времени и памяти, однако не гарантирует кратчайшего решения и в данном примере дал путь значительно большей стоимости, чем A*. Алгоритм A*, напротив, обеспечивает такой же кратчайший путь, как BFS, но делает это значительно эффективнее. Особенно хорошо это видно для комбинации A* с эвристикой h2: по сравнению с BFS стоимость пути остаётся той же, но временная сложность уменьшается с 90532 до 1442, а ёмкостная – со 114058 до 2267.

Таким образом, общий эксперимент показывает, что использование эвристической информации значительно повышает эффективность поиска. Среди методов информированного поиска жадный поиск имеет наименьшие временные и ёмкостные затраты, однако уступает алгоритму A* по качеству

найденного решения. Алгоритм A* обеспечивает меньшую стоимость пути, а наиболее эффективной его реализацией в данном эксперименте оказался вариант с эвристикой h2, поскольку при сохранении оптимальности решения он требует существенно меньших затрат времени и памяти, чем A* с эвристикой h1.